

Technical Cheat Sheet

Every term, model, method, dataset, and training decision in the project — defined and explained

1. Glossary of terms

Grouped by theme. Each entry: what it is, and how it is used in this project specifically.

1.1 Representations and spaces

Term	Definition and role in this project
Embedding / representation	A vector of numbers a model produces for an input, positioning it in a geometric space where distance ~ semantic dissimilarity. Everything in this project compares such vectors: 768-d hidden states (LLMs), 512-d MobileCLIP image vectors, 768-d SigLIP image/text vectors.
Activation / hidden state	The output of a specific layer for a specific input. LLMs produce one per token per layer; Experiment A stores one vector per passage per layer via mean pooling.
Mean pooling	Averaging token-level vectors over the sequence (masked to real tokens) to get one fixed-size vector per text. Lossy — discards word order — which makes Experiment A's similarity estimates conservative.
L2 normalization	Scaling a vector to unit length. After it, dot product = cosine similarity. All retrieval vectors in Experiment B are L2-normalized; the Core ML adapter normalizes its output.
Cosine similarity	Angle-based similarity in $[-1, 1]$, blind to vector length. The metric Qdrant search uses, hence the metric the adapter is judged by.
Anisotropy	Unequal spread of a space's variance across directions. MobileCLIP's and SigLIP's spaces differ in anisotropy — the reason rotation-only alignment (Procrustes) failed while full linear (ridge) succeeded.
Isomorphic (up to linear map)	Two spaces containing the same information in different coordinates, convertible by one matrix. The hypothesis asserts trained models' spaces are approximately this.
Permutation symmetry	Reordering a layer's neurons (with consistent rewiring) changes weights completely but not the function. The reason raw weights of independent models cannot be compared, averaged, or transplanted.

1.2 Measurement and statistics

Term	Definition and role in this project
Gram matrix	For n inputs, the $n \times n$ matrix of pairwise similarities as one model sees them — a coordinate-free fingerprint of the representation. CKA compares two models' Gram matrices.
CKA (Centered Kernel Alignment)	Similarity in $[0,1]$ between two representation sets, invariant to rotation, permutation, and uniform scaling — exactly the symmetries that leave a network's function unchanged. Used in A2 for every layer pair.
R^2 (coefficient of determination)	Fraction of the target's variance a prediction explains; 1 = perfect, 0 = as good as predicting the mean, negative = worse than the mean (seen in the starved run's random baseline). Used in A3 and B2 on held-out data.
Recall@K	Fraction of queries whose correct item appears in the top K results. The operational gate of Experiment B: adapter must keep $\geq 90\%$ of the ceiling's Recall.
Held-out / train-eval split	Fitting on one subset (75%) and measuring on untouched data (25%), so scores reflect generalization, not memorization. The same fixed split is reused across B2/B3/MLP for direct comparability.

Term	Definition and role in this project
Baseline (floor) / ceiling	The score with no learning (random weights; raw cross-space retrieval) vs the best achievable (native SigLIP). Every result in the project is read as its position between the two.
Sample-to-dimension ratio	Training rows per input dimension of a fitted map. At 1.6 (run 1) ridge is starved and systematically understates alignment; at ~10 (run 2) estimates stabilize. The project's key methodological lesson.
Overfitting / early stopping	Fitting noise beyond signal; prevented in the MLP comparison by keeping the checkpoint with best held-out cosine (reached at epoch 30 of 300).

1.3 Maps and training

Term	Definition and role in this project
Reparameterization (structural)	Folding a multi-branch training-time block (parallel convs + BatchNorm + identity) into a single equivalent convolution at inference. Used throughout MobileCLIP-S1; the reason its deployed graph is simpler and faster than its printed module tree, and why the ANE compiles it efficiently.
Identity normalization	A preprocessing Normalize step with mean 0 and std 1, i.e. no change. MobileCLIP-S1 uses it (raw [0,1] pixels); replicating the pipeline incorrectly on device silently invalidates the adapter's calibration.
Adapter (linear map)	Here: a single 512×768 matrix W translating MobileCLIP image vectors into SigLIP space: $y = L2norm(W \cdot x)$. 1.6 MB fp32 / 0.79 MB fp16. The project's deliverable.
Ridge regression	Linear least squares with an L2 penalty $\alpha \cdot I$ stabilizing the inverse: $W = (X^T X + \alpha I)^{-1} X^T Y$. Closed-form — solved exactly in one line, no gradient descent. Fits both the stitching maps (A3) and the adapter (B2) in seconds.
Closed-form solution	An answer computed by formula rather than iterative optimization. Why 'training' the adapter costs seconds and is exactly reproducible.
Orthogonal Procrustes	The best pure rotation aligning two point sets, from the SVD of $X^T Y$. Tests the stronger 'rigidly same shape' claim; succeeded implicitly in A (spaces near-rotational), failed in B ($R^2 = 0.038$) due to anisotropy differences.
MLP (multi-layer perceptron)	A small neural network (here 512->1024->768 with GELU) used as a controlled upgrade attempt. Its +0.003 non-improvement proved the linear map optimal.
GELU	A smooth activation function standard in transformers; the nonlinearity inside the comparison MLP.
Cosine loss	$1 - \cos(\text{prediction}, \text{target})$; trains the MLP directly on the quantity retrieval cares about, rather than MSE.
Model stitching	Connecting layer i of model A to layer j of model B through a small trained map and measuring the penalty. A3 is stitching with closed-form ridge and R^2 as the penalty measure.
Fine-tuning vs from-scratch	Fine-tuning updates an existing checkpoint (coordinates inherited); from-scratch starts from random init (coordinates independent). Both Experiment A models are from-scratch — the property that makes their alignment evidence of convergence.

1.4 Systems and deployment

Term	Definition and role in this project
Qdrant / vector database	A database that stores embeddings and answers nearest-neighbor queries. The target architecture: one collection, 768-d, cosine metric, written by both tiers.
HNSW	Hierarchical Navigable Small World — the graph index structure making approximate nearest-neighbor search fast; one shared graph instead of two is part of the efficiency payoff.

Term	Definition and role in this project
Core ML / .mlpackage	Apple's on-device inference format and runtime. B4 exports the adapter as an .mlpackage (MatMul + L2-normalize) via coremltools' PyTorch->MIL conversion.
Neural Engine (ANE)	The iPhone's NPU; runs fp16 Core ML models at very low energy. Executes both MobileCLIP and the adapter.
fp16 / fp32	16- vs 32-bit floating point. The adapter ships fp16 (0.79 MB, precision loss < 1e-3 — negligible for cosine ranking).
DATA_DIR convention	All pipeline stages read/write artifacts (activations.npz, pairs.npz, adapter.npz) under one configurable directory — on Colab, a mounted Google Drive path — making every stage stateless and VM-restart-proof.
Streaming / parquet datasets	Hub-hosted columnar data readable without custom loader scripts. The project migrated to parquet (wikitext) and direct official downloads (COCO) after HF deprecated script-based datasets mid-project.

2. Models: full profiles

Model	Architecture & size	Training data / objective	Role here
GPT-2 (124M)	Decoder-only transformer, 12 blocks, d=768, learned positional embeddings, GPT-2 BPE tokenizer (50,257 vocab). OpenAI, 2019.	WebText (~40 GB of Reddit-linked pages); next-token prediction.	Experiment A, model A. Frozen; only hidden states read.
Pythia-160M	GPT-NeoX decoder-only transformer, 12 blocks, d=768, rotary positional embeddings, NeoX tokenizer. EleutherAI, 2023.	The Pile (~825 GB, 22 diverse sources); next-token prediction. Trained from scratch — no shared ancestry with GPT-2.	Experiment A, model B. Frozen. The LM-head weight (embed_out) is discarded on load — the benign UNEXPECTED warning.
Random-weights Pythia	Identical architecture to Pythia-160M, weights freshly sampled from the init distribution; zero gradient steps.	None — that is the point.	The control: measures similarity obtainable from architecture + shared inputs alone. Trained-minus-random = effect of learning.
MobileCLIP-S1	Apple's efficient image-text model; hybrid conv-transformer image tower (FastViT/MC1), 21.5 M vision parameters verified by introspection; channel pyramid 64-128-256-512 with 4/12/20/4 blocks per stage; attention only in the final stage; embedding d=512. Designed for the ANE; loaded via open_clip (pretrained 'datacompdr').	Contrastive image-text training on DataCompDR (reinforced web image-text pairs).	The phone-tier encoder. Frozen; source space of the adapter.
SigLIP 2 (base, patch16-224)	Google's vision-language model; ViT-Base image tower (16px patches, 224px input), paired text tower; both embeddings d=768; ~1.5 GB checkpoint.	Sigmoid-loss contrastive image-text training (per-pair logistic objective instead of softmax) — the reason its absolute cosines run low (~0.15 diagonal) while ranking stays sharp.	The server-tier encoder and the target space. Frozen; its text tower produces all retrieval queries and is unaware the adapter exists.

Table: the five models. 'Frozen' = no weights updated anywhere in the project.

Encoders, precisely: an *encoder* is a model (or tower) mapping raw input to an embedding. The project uses four: GPT-2 and Pythia as text encoders (reading all 13 hidden-state levels, not just the last), MobileCLIP's image tower, and SigLIP's image + text towers. Decoding/generation is never used anywhere.

2.1 Inside MobileCLIP-S1 (verified by introspection)

Two structural facts matter for this project. **(a) Reparameterizable convolutions.** Nearly every block is a MobileOne/RepMixer unit holding parallel branches during training (3x3 conv + 1x1 conv + identity, each with BatchNorm) that fold algebraically into a single convolution at inference - training-time capacity at deployment-time cost. BatchNorm rather than LayerNorm throughout is what makes the fold possible. Attention appears only in the last stage (4 blocks, QKV 512 -> 1536) where the grid is already 8x8, so global mixing is nearly free. **(b) A one-layer bottleneck.** The trunk carries 1024 channels until a single final projection Linear(1024 -> 512) emits the embedding - so the capacity gap that limits R@1 is concentrated in one layer, and a wider-output sibling (MobileCLIP-B, MobileCLIP2) is the predicted remedy.

2.2 Preprocessing - a silent-failure trap

MobileCLIP-S1's transform is: bilinear Resize(256), CenterCrop(256x256), tensor conversion, and **Normalize(mean=(0,0,0), std=(1,1,1)) - an identity operation**. The model consumes raw [0,1] pixels, unlike standard CLIP models that subtract dataset means. Since the adapter matrix W is fitted against the embedding distribution this exact pipeline produces, replicating it inexactly on iOS (for example by copying conventional CLIP normalization constants) shifts every vector and degrades retrieval with no error raised. The iOS client must reproduce `resize-256 / center-crop / [0,1]` scaling and nothing else.

3. Data inputs: what, why, how

Dataset	Contents & size used	Why chosen	How used
WikiText-103 (Salesforce/wikitext, train split)	Long-form Wikipedia articles; 10,000 passages > 100 chars (run 2; run 1 used 1,646 validation passages).	Diverse, clean English prose; parquet-hosted (survives the datasets-scripts deprecation). Content is out-of-domain for neither model, favoring neither.	Identical passages fed to GPT-2, Pythia, and the random control; truncated at 64 tokens; batch 32; mean-pooled per layer -> activations.npz (3 × [13, 10000, 768]).
COCO val2017 (official cocodataset.org zips)	5,000 photos with 5 human captions each; 4,000 image-caption pairs used (first caption per image); ~1 GB images + 240 MB annotations.	Natural photos + human English captions — the adapter's actual production domain; standard retrieval benchmark, so results are sanity-checkable against published numbers; plain zips immune to loader-script deprecations.	Each image encoded by both image towers; each caption by SigLIP's text tower -> pairs.npz (mob_img 4000×512, sig_img 4000×768, sig_txt 4000×768), split 3000 train / 1000 eval.

Input processing details: LLM inputs are tokenized by each model's own tokenizer (deliberately different — part of the independence claim); attention-masked mean pooling excludes padding. Images are preprocessed by each encoder's own pipeline (resize/normalize per its training recipe); captions are tokenized by SigLIP's tokenizer, padded to 64. All embeddings are L2-normalized at extraction so every later stage works in cosine geometry.

4. Methods, step by step

4.1 CKA computation (A2)

For layer i of model A (matrix X , 10000×768) and layer j of model B (Y): center columns; $CKA = ||Y^T X||_F^2 / (||X^T X||_F \cdot ||Y^T Y||_F)$ — algebraically the cosine between the two centered Gram matrices. Computed for all 13×13 pairs, twice (trained-vs-trained, trained-vs-random). Read as: high diagonal = same content developed in the same order through depth.

4.2 Linear stitching (A3)

Per matched layer pair: 75/25 split; standardize inputs; solve ridge $W = (X^T X + aI)^{-1} X^T Y$ with $a=1.0$ on train; report variance-weighted R^2 on eval. Layer matching is one-to-one here (both models have 13×768 states); the code supports proportional matching for unequal depths. The run-1 -> run-2 comparison changed only N (1,646 -> 10,000) and the source split.

4.3 Adapter fitting (B2)

Ridge on raw (unstandardized, already L2-normalized) pairs with $a=0.01$: $W = (X^T X + aI)^{-1} X^T Y$, $X = \text{mob_img}[\text{train}]$ (3000×512), $Y = \text{sig_img}[\text{train}]$. Procrustes variant: zero-pad X to 768, $\text{SVD}(X^T Y) = U S V^T$, $R = U V^T$. Both evaluated on the same 1,000 held-out images by R^2 and mean cosine-to-target.

4.4 Retrieval evaluation (B3)

Queries: SigLIP text embeddings of the 1,000 eval captions (exactly the production query path). Galleries: (A) native sig_img — ceiling; (B) $L2\text{norm}(\text{mob_img} @ W)$ — the system; (C) raw mob_img zero-padded — chance floor. Similarity = dot product (all unit-norm); ground truth is the diagonal (caption k matched to image k); $\text{Recall}@\{1,5,10\}$ = fraction of captions ranking their own image in top K . Gate: $B \geq 90\%$ of A per K .

4.5 MLP comparison (B2-bis)

`nn.Sequential(Linear 512->1024, GELU, Linear 1024->768)`; AdamW lr $1e-3$, weight decay $1e-4$; cosine LR schedule; batch 256; 300 epochs; loss = $1 - \text{cosine}(\text{normalized prediction}, \text{target})$; early stopping keeps the best held-out-cosine checkpoint (epoch ~30). Same split as B2. Result: +0.003 cosine, retrieval unchanged -> linearity established.

4.6 Core ML export (B4)

Wrap W in a bias-free torch Linear plus `F.normalize`; `torch.jit.trace` on a dummy (1×512); `coremltools.convert` with FLOAT16 precision, iOS16 target; save Adapter.mlpackage (0.79 MB). Conversion path: PyTorch graph -> MIL ops -> mlprogram. Parity check on macOS: $\max |\text{CoreML} - \text{numpy}|$ expected $< 1e-3$ (fp16).

5. What was trained — and what deliberately was not

Component	Trained?	How / why not
GPT-2, Pythia, MobileCLIP, SigLIP 2	No — frozen throughout	The entire point: interoperability without touching the models.
Random-weights Pythia	No — never trained by design	Control condition isolating the effect of learning.
13 stitching matrices (A3)	Closed-form fit (seconds)	Ridge regression; 'trained' only in the loosest sense — no epochs, no gradients.
Adapter W (B2)	Closed-form fit (seconds)	Ridge on 3,000 pairs; refittable in seconds whenever either encoder is upgraded.
Procrustes R (B2)	Closed-form fit (seconds)	SVD; tests the rotation-only hypothesis.

Component	Trained?	How / why not
Comparison MLP (B2-bis)	Yes — 300 epochs, ~2 min GPU	The only gradient training in the project, run deliberately as a control to prove it was unnecessary.

Total compute for the full study — extraction, two A-runs, all fits, MLP, export — is under one GPU-hour on a free Colab T4.

Addendum — Experimental design vocabulary

Terms used when discussing why Experiment A was run the way it was (see report Appendix C).

Term	Meaning
Train / validation / test split	Partitions of a dataset defined by how much information reaches the model: train (weights fitted on it), validation (guides the experimenter's choices), test (used once, for the honest final number). The roles presuppose a model being trained - they carry no meaning for frozen models used only to read activations.
Probe stimuli	Inputs fed to a frozen model purely to elicit internal activations for measurement, analogous to a stimulus set in a perception experiment. WikiText plays this role in Experiment A: it trains nothing.
Held-out evaluation	Measuring a fitted map on rows it never saw. A 768x768 matrix with 590K free parameters can memorize a small sample exactly; in-sample R2 would then be meaningless. Every R2 in this project is held-out.
Samples per dimension	Training rows divided by input dimensionality - the practical measure of statistical power for a linear fit. Below roughly 5, ridge shrinkage dominates and estimates become systematically pessimistic. Run 1: 1.6. Run 2: 9.8.
Shrinkage bias (starved regression)	When rows are scarce relative to parameters, the ridge penalty forces coefficients toward zero to stay stable. The fit generalizes poorly in one direction: it understates the true relationship. Negative held-out R2 on a control is its signature - the fit generalizes worse than predicting the mean.
Pre-registration	Fixing success criteria before seeing results, so that a verdict cannot be chosen after the fact. Used for both experiments; Appendix C also records the one sub-threshold that was missed.
Diagnosed confound	A specific, checkable reason a measurement may be wrong, identified before rerunning and paired with a prediction that distinguishes it from the alternative. What separates a correction from fitting the result.

Rule of thumb. Before accepting any negative result from a fitted map, check that the measurement had the power to show a positive one: rows per parameter first, then alignment of the two data matrices, then the pooling or preprocessing that produced them.

Cheat Sheet Addendum - E and F series

Terms, models and settings introduced after the original cheat sheet was built. Everything in Sections 1-4 of that document still applies.

A1. Vision-transformer pooling: CLS, patches, and cls+patch

A vision transformer does not read an image as a whole. It cuts it into a grid of square **patches** - DINOv2 uses 14x14 pixels, so a 224x224 image becomes 16x16 = 256 patches - and treats each as a token, like a word in a sentence. One extra learned token, **CLS**, is prepended; it carries no image content of its own and exists to attend over all the patches and compress them into a single summary. After the transformer, the hidden state is [257, d]: row 0 is CLS, rows 1..256 are the patches, each describing its own region as informed by the whole image.

POOLING	What is taken	Width	Why
cls	row 0 only - the model's own global summary	d	the token the training objective operated on; one vector, cheapest
cls+patch	row 0 concatenated with the mean of rows 1..256	2d	DINOv2's own linear-probe protocol. The patch mean keeps texture, spatial statistics and fine detail that the CLS compression discarded; neither subsumes the other

Consequences in the notebooks. The width doubles, so N_PAIRS auto-scales to hold rows-per-input-dimension near 10 (dinov2-large with cls+patch: width 2048, 23,755 pairs, 11.1 rows/dim). And because pooling changes *what information* the vector carries - not merely how many dimensions hold it - it is a genuine confound for the scale sweep: every encoder in a sweep must use the same pooling, or a gain cannot be attributed to capacity. Measured example: dinov2-base with cls reached 40.9 per cent of ceiling and dinov2-large with cls+patch 58.3, but four factors differed between those runs, so the difference is not a capacity measurement.

A2. Regularization vs learning rate

Term	Where	What it does
alpha (ridge)	A3, B2, E-series: $W = (X^T X + \alpha I)^{-1} X^T Y$	Regularization strength - shrinks W toward zero before inversion. NOT a learning rate: there is no loop, no gradient and no schedule; the solve is closed-form
ALPHA_GRID	E-series	A search , not a schedule: each candidate is scored on a validation slice carved from TRAIN only, the winner is fixed, and one final fit is run on all training rows at that value
learning rate	F2 (1e-3, cosine-annealed), F3 (1e-4)	The genuine article - F trains artifacts by gradient descent. F3 uses a lower rate because it perturbs pretrained weights

Diagnostic value. How much alpha matters is itself a reading of the space: in the dinov2-large run every candidate scored 0.596 (22,755 rows against 2,048 dimensions is well-conditioned, so shrinkage is nearly irrelevant), while in E1.1 alpha = 10 produced R-squared -0.741 because GPT-2's near-degenerate space is severely ill-conditioned. Same grid, opposite sensitivity.

A3. Models added after the original sheet

Model	Architecture / size	Training	Role here
DINOv2 (small 22M / base 87M / large 304M)	ViT, patch 14; widths 384 / 768 / 1024 (doubled by cls+patch)	Self-supervised: self-distillation between two augmented crops plus masked-image modelling. No captions, no text tower, no cross-modal loss - text never enters the objective. Its dataset (LVD-142M) was curated using label signals, which biases which images were selected but never reaches the loss	The image arm of the E series, and the scale-sweep variable. Chosen precisely because a CLIP-family encoder would make the cross-modal test circular
bge-m3 (~568M)	XLM-RoBERTa backbone, 1024-d dense output, one size only	Contrastive retrieval training on ~1.2B multilingual text pairs; dense, sparse and multi-vector modes (only dense is used here). Never saw a pixel	The text arm of E1/E2. Its space is <i>shaped for cosine</i> , which E1.1 shows is not incidental
GPT-2 as a feature extractor	124M, 768-d, masked-mean pooled over ≤ 64 tokens	Next-token prediction only - never optimized for similarity. Used for inference, never generation	The E1.1 contrast arm: a text space with no similarity training. Effective rank 6.5 of 768, mean pair-cosine +0.999

A4. Terms introduced by the E/F results

Term	Meaning and measured anchor
matched ceiling	A jointly trained model scored on the same rows, same direction and same caption treatment as the system. Mismatching any of the three inflates the percentage: an averaged-caption system against a single-caption ceiling read 55.2 per cent where the matched figure is 40.9
caption averaging	COCO supplies five human captions per image; averaging their embeddings cancels annotator idiosyncrasy. R-squared 0.278 $\rightarrow$ 0.511, R@1 0.160 $\rightarrow$ 0.358. The ceiling must be averaged too (SigLIP: 0.648 single, 0.875 averaged)
metric degeneracy	A similarity metric can be uninformative in a given geometry. In GPT-2's space (pair-cosine +0.999) cosine reads 1.000 for a map whose retrieval runs at 9.5 per cent - the mirror of Experiment B, where R-squared understated a system whose cosine was honest
stitching penalty	$1 - R@K(\text{stitched}) / R@K(\text{ceiling})$: what a downstream consumer loses when fed a translated vector. Measured 0.049 at R@5 - near-transparent
relational loss	Matches the student's in-batch similarity (Gram) matrix to the teacher's, so training targets relational structure rather than coordinates. Used in F2/F3 alongside cosine
teacher-tagged checkpoint	A cache that records every setting affecting its contents (teacher id, encoder, pooling, width) and validates them on resume. Introduced after three separate silent failures from caches keyed on only part of the configuration
MATCH_DIM	Optional PCA projection of every encoder to a common width, so a scale sweep compares representation quality rather than fit capacity. Measured as a weak effect once N scales with width, since rows-per-parameter then stays constant

A7. What a transformer layer actually outputs (and what pooling does)

Loose phrasing to avoid: a layer does **not** produce 'a vector'. It produces **one vector per token**. A five-token passage through GPT-2 yields a grid of 13 layers x 5 tokens, each cell a 768-number vector - 65 vectors from one passage. Attention mixes the tokens within each layer (so the vector at position 'sang' carries context from 'the bird' before it), but nothing ever merges them into one; a transformer stage reads a set of token vectors and writes a set of the same shape.

Stage	Shape	What it is
layer output	[T, 768] per layer	one vector per TOKEN; T varies per passage
masked mean pooling	[768] per layer	average across the T real tokens, pads excluded from both the sum and the divisor. Done separately within each layer, so the layer axis survives - that axis is what Experiment A measures
one passage	13 x [768]	one vector per layer

Stage	Shape	What it is
the saved array	[13, 10000, 768]	13 layers x 10,000 passages x 768 coordinates. Row i is the SAME passage in GPT-2 and in Pythia

Worked masked mean. Take T = 3 real tokens plus one pad, and 4 coordinates instead of 768:

token	vector	mask
The	[2, 0, 4, 1]	1
bird	[0, 6, 2, 3]	1
sang	[4, 3, 0, 2]	1
<pad>	[9, 9, 9, 9]	0

Sum over real tokens only: $[2+0+4, 0+6+3, 4+2+0, 1+3+2] = [6, 9, 6, 6]$. Divide by the number of real tokens, 3, not by 4: **[2, 3, 2, 2]**. The pad row contributes nothing to the numerator and is excluded from the denominator - had it been included the result would be $[3.75, 4.5, 3.75, 3.75]$, contaminated by a token carrying no content.

Why pool at all. Two tokenizers do not agree on what the tokens are, so no token-to-token correspondence exists between GPT-2 and Pythia; passage-to-passage correspondence does. T also varies per passage, so per-token states do not form a rectangle, and both CKA and ridge need $[N, 768]$ matrices with row i the same passage in every model. Pooling moves the comparison to the level where alignment is real.

GPT-2 has no summary token. BERT and DINOv2 prepend a CLS token trained to represent the whole input; GPT-2 does not, because its attention is causal - a prepended token would see nothing, being first in line. Its only fully-contextual position is the last token, which over-weights the ending, so masked mean is the standard choice. Note the consequence: GPT-2 never produced a sentence-level vector during its own training, which is the literal sense in which it was 'never optimized for similarity'.

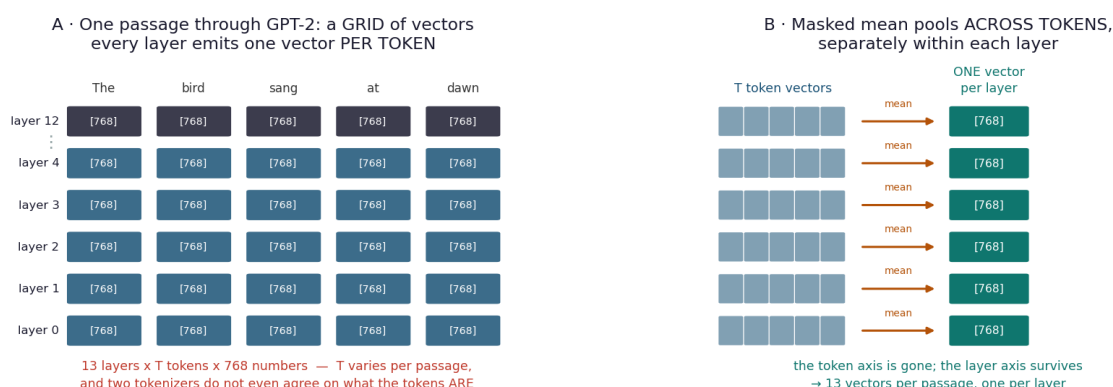


Figure A2: the token-by-layer grid, what masked mean collapses, and what each axis of the saved array means.

A6. Running the notebooks: runtime and storage

The notebooks run unchanged on Colab or on a local machine. Two independent choices, and they are often confused:

Choice	Options	What it controls
Runtime (where code executes)	Colab hosted VM (T4 / L4 / CPU) local kernel	Chosen in the Colab menu (Runtime -> Change runtime type), or simply by opening the file in Jupyter. No code selects it. To use your own GPU inside the Colab interface: install <code>jupyter_http_over_ws</code> , launch jupyter with <code>--NotebookApp.allow_origin='https://colab.research.google.com'</code> , and paste the printed <code>http://localhost:8888/?token=...</code> into Connect -> Connect to a local runtime
Storage (where data lives)	STORAGE = 'drive' (default) 'local' 'env'	'drive' mounts MyDrive/convergence_experiment - survives session timeouts, so checkpoints resume. 'local' uses LOCAL_DIR on whatever machine is running; on a hosted Colab VM that disk is ERASED at session end, so downloads restart from zero. 'env' honours a DATA_DIR already set. On a local kernel, 'drive' falls back to LOCAL_DIR automatically and says so - Drive mounting is a Colab-VM feature and cannot work locally

Which to use. The dominant cost in the E-series is downloading COCO images, not GPU time: a hosted VM has datacenter bandwidth, a local machine has your uplink, so hosted can finish the fetch faster even with a weaker GPU. Against that, a local kernel has no quota and no session timeout - which matters most for the largest sweep point (dinov2-large at cls+patch, 23,755 images). Because checkpoints are plain .npz validated against the current configuration, the two can be mixed: fetch on a hosted runtime writing to Drive, then continue locally with STORAGE='env' pointed at the same folder synced by Google Drive for Desktop.

A free T4 is sufficient for dinov2-small and -base; CPU works but turns minutes into hours, and the first cell prints a warning when no GPU is found. The cell also prints which environment, device and DATA_DIR it resolved, which is the quickest way to confirm a local runtime actually took effect - it reports 'local machine' rather than 'Colab'.

A5. What is and is not trained

Series	Trained?	Detail
A, B, E	No	Models are downloaded frozen and run under <code>no_grad</code> ; the only fitted object is a closed-form ridge matrix outside both models. Weights are bit-identical to the downloads afterwards - delete W and nothing remains
F2	Yes	Trains a separate MLP head <i>beside</i> the frozen encoders; the encoders themselves are untouched
F3	Built, deliberately unrun	The only notebook that would modify a pretrained model's own weights (MobileCLIP's last stage). Gated on F2's verdict and on whether moving the ceiling is worth forfeiting the seconds-long refit